How do we address it

- We could develop a more involved operational semantics?
- More complicated, may still end up being overly restrictive or permissive without a means to measure that quality (there feels like there should be something more there)
- Can we take inspiration from the way others have formulated these designs, namely weak memory models

An Axiomatic Concurrency Model

```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

update(src, +100)

update(src, -100)

Candidate executions are tuples of $(\mathbb{E}, po, co)$

$\mathbb{E}$ is the set of program events

po is the program order representing the order in which a behaviour spawns subsequent behaviours

co is the cown order representing the order between one behaviour completing and subsequent behaviour running